



WHITE-BOX PERFORMANCE AUDIT

Performance Rapport

Rembro Webshop · Node.js + Express + React SPA + SQLite · Juni 2026

TYPE	DATUM	BRANCH
Statische analyse + build	Juni 2026	develop

Gemeten baseline: Vite build 1.41s · Bundle 196 KB (62 KB gzip)
17 bevindingen · 4 Hoog · 7 Gemiddeld · 5 Laag · 1
Architectureel

0
CODE
SPLITS

0
CACHE
HEADERS

1
JS
CHUNK

17
BEVINDINGEN

1. Baseline metrics

Gemeten op 16 juni 2026 · Branch: develop · Commit: 35b6508 · npm run build + statische code-analyse

GESCHATTE CORE WEB VITALS (CSR SPA ZONDER SSR)



LCP

~1.5–3.5 s*

Drempel: < 2.5 s



CLS

~0.1–0.4*

Drempel: < 0.1



INP

~50–200 ms*

Drempel: < 200 ms



TTFB

~5–30 ms (server)

JS-afhankelijk voor content

* Geschat — vereist Lighthouse/WebPageTest op een live URL voor exacte waarden. Zie sectie 7.

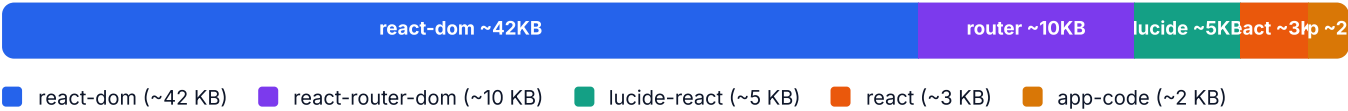
 **VITE BUILD (GEMETEN)**

JS bundle (raw)	196.85 KB
JS bundle (gzip)	62.42 KB
CSS bundle (raw)	14.94 KB
CSS bundle (gzip)	3.68 KB
Aantal JS chunks	1 (geen splits)
Modules getransformeerd	1597
Build-tijd	1.41 s

 **DATABASE (GEANALYSEERD)**

Queries / GET /products	1
Queries / GET /products/:id	1
Queries / POST /orders (3-item)	13
Queries / POST /orders (N-item)	4 + 4N
Index op order_items.order_id	Ontbreekt
Index op orders.created_at	Ontbreekt
Compressie-middleware	Geen

GESCHATTE BUNDEL-SAMENSTELLING (196 KB GZIP: 62 KB)



ROUTE-KAART

Route	Rendering	Data-bron	API-calls bij laden
/	CSR	GET /api/products (ALL)	1 
/shop	CSR	GET /api/products (ALL)	1  dubbel
/product/:id	CSR	GET /api/products/:id	1
/winkelwagen	CSR	localStorage	0
/afrekenen	CSR	POST /api/orders bij submit	0 (1 bij submit)
/contact	CSR	Geen	0

Route	Rendering	Data-bron	API-calls bij laden
/admin	CSR	GET /api/products + GET /api/orders	2 (parallel)

2. Alle bevindingen

Gesorteerd op ernst en impact. Effort = inspanning voor implementatie. Impact = verwachte prestatiewinst voor eindgebruiker.

ID	Ernst	Titel	Effort	Categorie
PER-001	HOOG	Geen code splitting — 196 KB in één chunk	Gemiddeld	Bundle
PER-002	HOOG	Dubbele product-fetch (Home + Shop)	Laag	Fetching
PER-003	HOOG	N+1 queries in POST /orders	Laag	Database
PER-004	HOOG	Ontbrekende index op order_items.order_id	Laag	Database
PER-005	HOOG	Geen HTTP-caching op GET /api/products	Laag	Caching
PER-006	HOOG	Statische assets zonder far-future cache headers	Laag	Infra
PER-007	GEMIDDELD	Admin HMAC token herberekend per request	Laag	Backend
PER-008	GEMIDDELD	SELECT * in orderverwerking	Laag	Database
PER-009	GEMIDDELD	HomePage haalt alle producten op voor 3 items	Laag	Fetching
PER-010	GEMIDDELD	Afbeeldingen zonder dimensies/lazy-load/LCP-hint	Laag	Rendering
PER-011	GEMIDDELD	CartContext herrendert alle consumers bij elke mutatie	Gemiddeld	Rendering
PER-012	GEMIDDELD	DatabaseSync blokkeert event loop bij elke query	Hoog	Infra
PER-013	GEMIDDELD	Admin herlaadt alle data na elke product-mutatie	Laag	Fetching
PER-014	LAAG	Intl.NumberFormat per aanroep geconstrueerd	Laag	Bundle
PER-015	LAAG	Geen CORS preflight-caching (max-age)	Laag	Backend
PER-016	LAAG	Admin refresht orders na product-save/delete	Laag	Fetching
PER-017	ARCHITECTUREEL	Geen SSR — alle content afhankelijk van JS-executie	Hoog	Rendering

SNEL TE IMPLEMENTEREN + HOGE IMPACT

PER-002 Dubbele fetch

PER-003 N+1 queries

PER-004 DB index

PER-005 API caching

PER-006 Static caching

PER-010 Afbeeldingen

GEMIDDELDE INSPANNING + HOGE IMPACT

PER-001 Code splitting

PER-011 Context splits

SNEL + LAGE IMPACT (QUICK WINS)

PER-007 HMAC cache

PER-014 Intl cache

PER-013 Admin refresh

PER-016 Gerichte fetch

PER-009 Limit param

HOGE INSPANNING (STRATEGISCH)

PER-012 Async DB

PER-017 SSR toevoegen

3. Hoge bevindingen

HOOG

PER-001 Geen code splitting — hele applicatie in één 196 KB JS-chunk

Effort: Gemiddeld

LOCATIE	client/src/App.jsx:3-11 · vite.config.js
GEMETEN	196.85 KB JS (62.42 KB gzip) · 1597 modules getransformeerd · 1 chunk
IMPACT	Elke bezoeker van de homepage downloadt, parseert en compileert admin-paneel, checkout-logica en alle paginacomponenten — ook al gebruikt hij er maar één. Parse + compile op mid-range mobiel: ~200–400 ms extra.
HERSTEL	manualChunks in vite.config.js + React.lazy() + Suspense in App.jsx . Verwacht resultaat: initial chunk ~65–80 KB (gzip); AdminPage laadt alleen bij navigatie naar /admin.

```
// vite.config.js - HERSTEL
export default defineConfig({
  plugins: [react()],
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          "vendor-react": ["react", "react-dom", "react-router-dom"],
          "vendor-lucide": ["lucide-react"]
        }
      }
    }
  }
});

// client/src/App.jsx - HERSTEL
const AdminPage = lazy(() => import("../pages/AdminPage.jsx"));
const CheckoutPage = lazy(() => import("../pages/CheckoutPage.jsx"));
// ... overige pagina's ook lazy
<Suspense fallback=<LoadingState />>
  <Routes>... </Routes>
</Suspense>
```

HOOG

PER-002

Dubbele product-fetch — HomePage én ShopPage roepen GET /api/products onafhankelijk aan

Effort: Laag

LOCATIE

`client/src/hooks/useProducts.js:1-28` · `HomePage.jsx:10` · `ShopPage.jsx:8`

IMPACT

Home → Shop navigatie = 2× network round-trip + 2× SQLite query voor identieke data. Elke fetch: +100–300 ms RTT. Totaal: 200–600 ms onnodige latentie op de user journey.

HERSTEL

Module-level cache in `useProducts.js` — eerste aanroep fetcht, tweede aanroep leest uit module-scope variabele. Optioneel: React Query/SWR voor invalidatie.

```
// client/src/hooks/useProducts.js — HERSTEL
let cachedProducts = null;
let pendingRequest = null;

export function useProducts() {
  const [products, setProducts] = useState(cachedProducts ?? []);
  const [loading, setLoading] = useState(cachedProducts === null);

  useEffect(() => {
    if (cachedProducts === null) return; // cache hit
    if (!pendingRequest) {
      pendingRequest = api.getProducts().then(d => { cachedProducts = d; return d; });
    }
    pendingRequest.then(d => { setProducts(d); setLoading(false); });
  }, []);
}
```

HOOG

PER-003

N+1 query pattern in POST /orders — één SELECT per winkelwagen-item

Effort: Laag

LOCATIE

`server/routes/orders.js:83-84`

GEMETEN

POST /orders met 3 items = 13 SQLite queries; met 10 items = 4 + 4×10 = 44 queries, alle synchrone event-loop-blokkering.

IMPACT

Elk extra winkelwagen-item voegt 1 extra synchrone DB-call toe. Simultane checkouts wachten achter elkaar in de single-threaded event loop.

```
// orders.js:83-84 — PROBLEEM
const products = productIds.map((id) => {
  db.prepare("SELECT * FROM products WHERE id = ?").get(id) // N queries!
});

// HERSTEL — één query met IN-clause
const placeholders = productIds.map(() => "?").join(", ");
const rows = db.prepare(
  `SELECT id, name, price, stock FROM products WHERE id IN (${placeholders})`
).all(...productIds);
const productMap = new Map(rows.map(p => [p.id, p]));
```

HOOG

PER-004

Ontbrekende index op order_items.order_id — full table scan bij elke orderfetch

Effort: Laag

LOCATIE

server/db/database.js:57-68

IMPACT

SQLite maakt GEEN automatische index bij FOREIGN KEY. Met 10.000 orders (3 items gem.) = 30.000 rijen gescand per getOrderWithItems()-aanroep. Linaire degradatie $O(n)$ i.p.v. $O(\log n)$.

```
// server/db/database.js — HERSTEL (voeg toe na CREATE TABLE)
db.exec(`
  CREATE INDEX IF NOT EXISTS idx_order_items_order_id
    ON order_items(order_id);
  CREATE INDEX IF NOT EXISTS idx_orders_created_at
    ON orders(created_at DESC);
  CREATE INDEX IF NOT EXISTS idx_products_created_at
    ON products(created_at DESC);
`);
```

HOOG

PER-005 + PER-006

Geen HTTP-caching op API én geen far-future cache op statische assets

Effort: Laag

LOCATIE

server/routes/products.js:36 · server/index.js:19, 38

IMPACT

Elke bezoeker downloadt de 196 KB JS-bundle opnieuw. Vite-assets zijn content-hashed (veilig voor eindeloze cache) maar express.static() heeft standaard max-age=0. Terugkerende bezoekers besparen 0 bytes.

```
// server/index.js — HERSTEL
import compression from "compression"; // npm i compression
app.use(compression()); // gzip alle responses
app.use(express.static(distPath, {
  maxAge: "1y", immutable: true // hashed assets = eeuwige cache
}));

// server/routes/products.js — Cache-Control voor productdata
res.set("Cache-Control", "public, max-age=30, stale-while-revalidate=300");
res.json(products);
```

4. Gemiddelde & lage bevindingen

GEMIDDELD

PER-007 Admin HMAC-token herberekend bij elke authenticated request

Effort: Laag

LOCATIE `server/utils/auth.js:8-13, 22`

IMPACT `crypto.createHmac(...).digest()` per request — token is een pure functie van env-vars die nooit wijzigen. Onnodige CPU op de event loop thread, hoewel HMAC-SHA256 snel is (~1–5 µs).

```
// HERSTEL — bereken één keer bij module-load
const ADMIN_TOKEN = crypto.createHmac("sha256", adminSecret)
  .update(`${adminUser}:${adminPassword}`).digest("hex");
export const createAdminToken = () => ADMIN_TOKEN; // geen berekening meer
```

GEMIDDELD

PER-009 HomePage haalt ALLE producten op om er 3 te tonen

Effort: Laag

LOCATIE `client/src/pages/HomePage.jsx:10-11` · `server/routes/products.js:36`

IMPACT Huidig (6 producten): verwaarloosbaar. Bij 1.000 producten: 1.000 rijen geserialiseerd, overgedragen, geparsed — 997 weggegooid. Herstel: `?limit=3` query-parameter op de API.

GEMIDDELD

PER-010 Afbeeldingen zonder dimensies, lazy loading of LCP-prioriteitshint

Effort: Laag

LOCATIE `client/src/components/ProductCard.jsx:13` · `HomePage.jsx:22` · `ProductPage.jsx:29`

IMPACT Geen width/height → browser kan geen ruimte reserveren → CLS bij elke afbeelding. Hero-afbeelding (1400px Unsplash, ~150–300 KB) heeft geen `fetchpriority="high"` → concurreert met JS-bundle voor bandbreedte.

```
// HomePage.jsx hero — HERSTEL
<img src={heroUrl} alt="..." width="1400" height="933"
  fetchpriority="high" decoding="async" />

// ProductCard.jsx — HERSTEL
<img src={product.image} alt={product.name}
  width="900" height="600" loading="lazy" decoding="async" />
```

GEMIDDELD

PER-011 CartContext herrendert alle consumers bij elke winkelwagen-mutatie

Effort: Gemiddeld

LOCATIE `client/src/context/CartContext.jsx:17-65`

IMPACT Toevoegen aan winkelwagen herrendert: Layout (header) + elke ProductCard op de pagina. Bij 6 producten: 8 onnodige renders. Bij 50+ producten: 52 renders. Oplossing: Context splitsen in items-context en actions-context.

GEMIDDELD

PER-012

DatabaseSync blokkeert Node.js event loop bij elke SQLite-query

Effort: Hoog

LOCATIE

`server/db/database.js:14` · `server/routes/orders.js:82-131`

IMPACT

Elke SQLite-aanroep blokkeert de V8 main thread. Simultane requests wachten achter elkaar. POST /orders (13 queries) blokkeert ~5–20 ms per checkout. Bij 10 gelijktijdige bestellingen: laatste wacht ~100–200 ms puur op DB-toegang.

OPTIES

1. Korte termijn: N+1 reduceren (PER-003 — meeste winst). 2. Medium: worker_threads voor DB-toegang. 3. Lang: PostgreSQL + async pg-driver.

LAAG

PER-014

Intl.NumberFormat per aanroep geconstrueerd — geen cached instantie

Effort: 2 min

LOCATIE

`client/src/utils/format.js:1-6`

HERSTEL

Hois new Intl.NumberFormat(...) naar module-scope. Admin-pagina rendert 20+ prijzen per render; elke aanroep maakt en gooit een formatter-object weg.

LAAG

PER-016 + PER-013

Admin herlaadt alle data (producten + orders) na elke product-mutatie

Effort: 10 min

LOCATIE

`client/src/pages/AdminPage.jsx:147, 163`

HERSTEL

Na saveProduct() en deleteProduct(): vervang loadAdminData() door gerichte getProducts()-call. Orderdata verandert niet bij product-mutaties.

ARCHITECTUREEL

PER-017

Pure CSR SPA — alle content afhankelijk van JS-executie (TTFB → LCP: 350ms–2.3s)

Effort: Hoog

LOCATIE

`server/index.js:38-46` · `client/src/main.jsx:8-16`

IMPACT

Initiële HTML = leeg <div id="root">. LCP-keten: HTML → JS download → JS parse (200–400 ms) → React hydrate → API fetch → render. Op 3G mobiel: tot 2.3 s voor eerste zichtbare content.

KORTE
TERMIJN

Voeg <link rel="preload" as="fetch" href="/api/products?limit=3"> toe aan index.html + <link rel="preconnect" href="https://images.unsplash.com">.

LANGE
TERMIJN

Overweeg migratie naar framework met SSR-ondersteuning (Next.js, Remix) voor statische pre-rendering van shop- en productpagina's.

5. Prioriteitenkaart

Gerangschikt op impact × implementatiesnelheid. Bovenste 5 items leveren de grootste prestatiewinst met de minste inspanning.

- 1

PER-004 — Voeg ontbrekende DB-indexes toe
3 regels SQL in database.js · Wegneming van O(n) table scans · Kritisch bij groei voorbij 1.000 orders

10 min
- 2

PER-006 — Far-future cache op statische assets
express.static({ maxAge:"1y", immutable:true }) · 196 KB bundle: 0 bytes herdownload voor terugkerende bezoekers

10 min
- 3

PER-005 — Gzip + Cache-Control op API-responses
npm i compression + 5 regels · Browser hergebruikt productdata 30s · Elimineer onnodige network round-trips

30 min
- 4

PER-003 — Fix N+1 queries in POST /orders
Vervang .map() + N SELECTs door één IN-query · 13 queries → 7 queries voor 3-item order · Event-loop blokkering halveert

20 min
- 5

PER-010 — Voeg afbeeldingsdimensies + lazy loading toe
width/height op alle img-tags · fetchpriority="high" op hero · loading="lazy" op product cards · CLS van ~0.3 → < 0.1

30 min
- 6

PER-002 — Module-level cache in useProducts.js
~20 regels code · Home→Shop navigatie: -200 tot -600 ms wachttijd · Elimineert dubbele network + DB round-trip

30 min
- 7

PER-014 — Cache Intl.NumberFormat instantie
1 regel — verplaats naar module-scope in format.js · Elimineert object-allocatie bij elke prijs-render

2 min
- 8

PER-001 — Code splitting + React.lazy voor routes
vite.config.js + App.jsx herschrijven · Initial chunk: 196 KB → ~65 KB (gzip) · AdminPage laadt alleen bij /admin-bezoek

60 min

VERWACHTE PRESTATIEVERBETERING NA QUICK WINS 1-7 (±2 UUR WERK)

DB queries/order

13 → 7

-46% event-loop blokkering

Repeat visit assets

197KB → 0KB

100% cache hit (far-future)

Home→Shop fetches

2 → 1

-200-600ms navigatie

CLS score

~0.3 → <0.1

"Goed" drempel bereikt

6. Loadtest-plan & k6-scripts

Voer tests uit op een lokale of staging-omgeving. Nooit op productie. Installeer k6: `winget install k6` (Windows) of `brew install k6` (Mac).

Starten: `node server/index.js` (poort 3001) → in tweede terminal: `k6 run scripts/k6-browse.js`

// scripts/k6-browse.js – Scenario 1: Browse (home → shop → productdetail)

```
import http from "k6/http";
import { sleep, check } from "k6";

export const options = {
  stages: [
    { duration: "30s", target: 10 }, // opwarmen
    { duration: "1m", target: 50 }, // piekbelasting
    { duration: "30s", target: 0 }, // afkoelen
  ],
  thresholds: {
    "http_req_duration": ["p(95)<500"], // p95 < 500ms
    "http_req_failed": ["rate<0.01"], // foutrate < 1%
  },
};

export default function () {
  // 1. Haal productlijst op (= HomePage + ShopPage)
  const r1 = http.get("http://localhost:3001/api/products");
  check(r1, { "products 200": r ⇒ r.status === 200 });
  sleep(1);

  // 2. Open een productpagina (Urban Rugzak, slug of id)
  const r2 = http.get("http://localhost:3001/api/products/urban-rugzak");
  check(r2, { "product 200": r ⇒ r.status === 200 });
  sleep(2);
}
```

// scripts/k6-checkout.js – Scenario 2: Checkout (bestelling plaatsen)

```
import http from "k6/http";
import { sleep, check } from "k6";

export const options = {
  stages: [
    { duration: "30s", target: 5 },
    { duration: "1m", target: 20 }, // checkout = zwaarder
    { duration: "30s", target: 0 },
  ],
  thresholds: {
    "http_req_duration{name:checkout}": ["p(95)<800"],
    "http_req_failed": ["rate<0.01"],
  },
};

export default function () {
  const payload = JSON.stringify({
    customerName: "Test Klant", email: "test@example.com",
    phone: "0612345678", address: "Teststraat 1",
    postalCode: "1234AB", city: "Amsterdam",
    items: [{ productId: 1, quantity: 1 }],
  });
}
```

```
const r = http.post(
  "http://localhost:3001/api/orders", payload,
  { headers: { "Content-Type": "application/json" },
    tags: { name: "checkout" } }
);
check(r, { "order 201": r => r.status === 201 });
sleep(3);
}
```

// scripts/k6-spike.js – Scenario 3: Spike test (plotselinge piek, bijv. sale)

```
export const options = {
  stages: [
    { duration: "10s", target: 5 }, // rustig
    { duration: "10s", target: 200 }, // plotselinge piek
    { duration: "1m", target: 200 }, // piek sustained
    { duration: "10s", target: 5 }, // terugval
  ],
  thresholds: { "http_req_failed": ["rate<0.05"] }, // max 5% fouten
};
// Zelfde request-body als k6-browse.js
```

AANBEVOLEN DREMPELWAARDEN (THRESHOLDS)

Metric	Drempel	Toelichting
http_req_duration p(95)	< 500 ms	Browse-endpoints (GET products, GET product/:id)
http_req_duration p(95) checkout	< 800 ms	POST /orders (13 synchrone DB-queries)
http_req_failed rate	< 1%	HTTP 5xx + netwerk-errors
http_req_duration p(99)	< 2 s	Absolute bovengrens — spiketests
Minimale checkout-doorvoer	> 10 req/s	Bij piek van 50 gelijktijdige gebruikers

7. Monitoring aanbevelingen

Continue meting na implementatie van de verbeteringen — prestatie is geen eenmalige actie.



Core Web Vitals (RUM) — Voeg web-vitals npm-pakket toe en stuur LCP/CLS/INP naar een gratis endpoint (bijv. Cloudflare Analytics of simpele beacon-call). Meet op echte gebruikersapparaten, niet alleen in Lighthouse.



Trage queries loggen — Voeg query-timing logging toe in `withTransaction()` en route-handlers: log queries die langer dan 50 ms duren naar `console.warn`. Zo vang je productie-degradatie vroeg op.



Bundle-size budget in CI — Voeg `bundleSize` of Vite's `build.chunkSizeWarningLimit` toe aan de build. Stel budget in: initial chunk $\leq$ 75 KB gzip. GitHub Actions breekt de build als het budget wordt overschreden.



API-latentie monitoring — Voeg een Express-middleware toe die request-duur meet en logt (p50/p95). Na 1.000 verzoeken heb je een betrouwbaar latentieprofiel per endpoint.



Lighthouse CI in pull requests — Installeer `@lhci/cli` en voer Lighthouse uit op elke PR. Stel drempelwaarden in: Performance $\geq$ 80, LCP $\leq$ 2.5 s. Breek de CI bij regressie.



SQLite database-grootte — Monitor de bestandsgrootte van `server/data/webshop.sqlite` wekelijks. Bij > 100 MB is het tijd voor een achiverings- of migratiestrategie naar PostgreSQL.

8. Vereist handmatige verificatie

De volgende metingen vereisen een draaiende server, een browser of een externe testomgeving.



Exacte Core Web Vitals — Voer Lighthouse of PageSpeed Insights uit op een live URL voor gemeten LCP/CLS/INP/TTFB. Alle waarden in dit rapport zijn schattingen op basis van code-analyse.



SQLite query-tijden — Voer `EXPLAIN QUERY PLAN SELECT ...` uit op de productie-database om de werkelijke query-plannen te controleren, met name voor `getOrderWithItems()`.



Event-loop blokkering — Gebruik `clinic.js flame` of `node --inspect` met een load test om te meten hoeveel ms de event loop daadwerkelijk wordt geblokkeerd door `DatabaseSync`.



Bundel-samenstelling — Voer `npx vite-bundle-visualizer` uit na de build voor exacte bibliotheek-per-bibliotheek groottes (niet de schattingen in dit rapport).



Werkelijke LCP-element — Open de app in Chrome DevTools → Performance-tab → neem een opname. Het LCP-element (hero-afbeelding of eerste productkaart) is apparaat- en viewport-afhankelijk.



Gzip-effect op API-responses — Na installatie van compression: controleer respons-headers op `Content-Encoding: gzip` en vergelijk payload-groottes voor en na, met `curl -H "Accept-Encoding: gzip" -v ....`



Cache-Control effectiviteit — Controleer na toevoeging van cache headers of Chrome DevTools "from cache" / "304 Not Modified" toont bij herhaalbezoeken voor statische assets.



Checkout-doorvoer onder concurrent load — Voer k6-checkout.js uit met 20+ gelijktijdige VUs en meet de werkelijke p95-latentie. SQLite's single-writer lock maakt dit het knelpunt bij hoge load.



REMBRO WEBSHOP

Performance Audit 2026

Dit rapport is gebaseerd op statische broncode-analyse en Vite build-output.
Alle CWV-waarden zijn schattingen — meet op een live omgeving voor exacte data.

BEVINDINGEN

17

HOOG

6

QUICK WINS

8

TIJDSINVESTERING

±4u